

9. Recursive Language Models (RLM)

Full study notes: every slide + every notebook cell, explained in plain English for someone who has never touched AI before.

Diagrams are written in **Mermaid** — in Notion, paste them into a `/code` block and set the language to **Mermaid** to see the picture.

Part A — Stuff you need to know first (the absolute basics)

Before any of the slides make sense, here are the five ideas everything else is built on. Read this part slowly; the rest is easy after it.

1. What is an LLM?

LLM = **Large Language Model**. It's a computer program trained on a giant pile of text. You give it some text, and it predicts what text should come next. ChatGPT, Claude, and Gemini are all LLMs. That's the whole trick — very fancy autocomplete.

2. What is a "prompt" and a "token"?

- A **prompt** is whatever you type into the model (your question, plus any documents you paste in).
- The model doesn't read letters or whole words. It chops text into small pieces called **tokens** (a token is roughly $\frac{3}{4}$ of a word). "engineer" might be 1–2 tokens.

3. What is the "context window"?

The **context** is everything the model can look at in one go — your question *plus* any data you give it. The model has a **maximum size** for this, called the **context window** (e.g. "128,000 tokens"). Think of it as the size of the desk the model is allowed to spread papers on. If your papers don't fit on the desk, you're stuck.

4. What is "attention"?

"Attention" is the internal mechanism the model uses to connect every token to every other token, so it can understand relationships ("this word refers to that

earlier word"). It's powerful — but it gets **slow and expensive** as the text grows, because every token has to be compared with every other token.

5. What is "context rot" (a.k.a. "lost in the middle")?

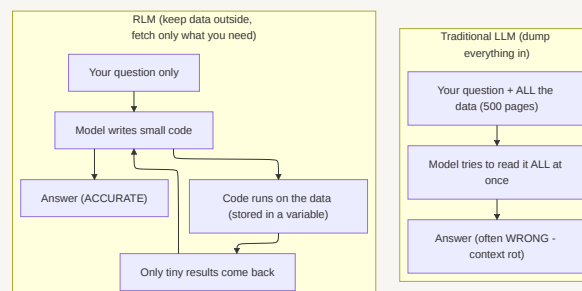
When you cram a huge amount of text into the model at once, it starts making mistakes — especially **missing things buried in the middle**. It's like being handed a 500-page book and asked to instantly remember one number on page 247. Humans can't do that in one read, and neither can LLMs. **This single problem is what the whole class is about.**

💡 **The one-line takeaway:** Big context = the model gets confused and misses things. Every tool in this class is a different clever way to *avoid* dumping everything into the context at once.

flowchart TB

```
subgraph T["Traditional LLM (dump everything in)"]
  A1["Your question + ALL the data (500 pages)"] --> A2["Model tries to read it ALL at once"]
  A2 --> A3["Answer (often WRONG - context rot)"]
end

subgraph R["RLM (keep data outside, fetch only what you need)"]
  B1["Your question only"] --> B2["Model writes small code"]
  B2 --> B3["Code runs on the data (stored in a variable)"]
  B3 --> B4["Only tiny results come back"]
  B4 --> B2
  B2 --> B5["Answer (ACCURATE)"]
end
```



Part B — The Slides, one by one

Slide 1 — Title: "Recursive Language Models"

Just the opening slide. It promises two things: (1) a new, cutting-edge way to handle context, and (2) a real-world debate (Claude Code vs Cursor) that proves *why* this matters in practice.

Slide 2 — RLM vs Traditional LLM

This slide contrasts the **old way** and the **new way**.

- **Traditional LLM:** Your prompt *is* the input. Every token goes through attention. Result → **context rot happens** (it gets confused with big inputs).
- **RLM:** The prompt (the huge data) is stored as a **variable**, like a file sitting in memory. The model interacts with it through **code** instead of reading it directly. Result → **no context rot, and it scales almost infinitely**.

The paper is by **Alex Zhang, Tim Kraska & Omar Khattab (MIT CSAIL)**.

💡 **Remember:** Old way = data goes *inside* the model. New way = data stays *outside* in a variable, and the model pokes at it with code.

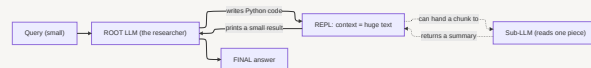
Slide 3 — How RLM Works (the architecture)

RLM has three parts:

1. **ROOT LLM** — the "boss" or "researcher." It only ever sees the **question**, never the giant data. Its job is to decide *how* to find the answer.
2. **REPL Environment** — REPL just means "a place where code runs" (like a Python notebook). The huge data is stored here as a variable called `context`. The Root LLM writes code, the REPL runs it, and sends back the small result.
3. **Available tools** the model can use inside the code:
 - `context[:1000]` → **Peek** at the first 1000 characters.
 - `re.findall(...)` → **Search** for a pattern.
 - `context[a:b]` → **Slice** out a specific chunk.
 - `rlm_agent(q, chunk)` → **Recurse** (hand a chunk to a smaller helper-LLM).

The whole thing runs in a loop: **Write code** → **Execute** → **See output** → **Iterate**. It's the same pattern as an AI "agent," except the *tool* the agent is using is *its own data*.

```
flowchart LR
    Q["Query (small)"] --> Root["ROOT LLM (the researcher)"]
    Root -->| "writes Python code" | REPL["REPL: context = huge text"]
    REPL -->| "prints a small result" | Root
    REPL -->| "can hand a chunk to" | Sub["Sub-LLM (reads one piece)"]
    Sub -->| "returns a summary" | REPL
    Root --> Ans["FINAL answer"]
```



Slide 4 — Step-by-step example

A worked example. **Question:** "What was ACME Corp's Q3 APAC revenue?" **Data:** a 500-page annual report stored as a variable.

1. **Initial Peek** — model reads `context[:2000]` to see the table of contents and how the document is laid out.
2. **Targeted Search** — runs `re.findall(r'APAC|Asia Pacific', context)` to find *where* the relevant pages are.
3. **Context Slicing** — slices out just `context[45000:52000]` and hands that small piece to a Sub-LLM to extract the exact Q3 figure.
4. **Final Synthesis** — the Root LLM takes the answer from the sub-agent and writes the final reply.

The punchline: the Root model never read all 500 pages. It only ever looked at tiny slices → **zero context rot**.

```
flowchart LR
    S1["1. Peek context[:2000] -> see structure"] --> S2["2. Search 'APAC' -> find location"]
```

```

S2 --> S3["3. Slice context[45000:52000] -> ask Sub-LLM"]
S3 --> S4["4. Synthesize -> final answer"]

```



Slide 5 — The Power of Recursion

"Recursion" = a thing that calls a smaller copy of itself. Here, a big task is split into chunks; each chunk can be split again into smaller chunks, each handled by its own Sub-LLM. Results flow back up to the top.

It mirrors how a **human researches**:

1. **Scan** the table of contents.
2. **Narrow** down to relevant chapters.
3. **Deep-dive** into specific sections.
4. **Synthesize** everything into an answer.

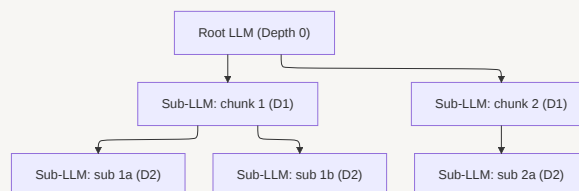
Because each helper only sees a small, focused piece, the data could be **10 million or 100 million tokens** — it doesn't matter. You just recurse deeper.

flowchart TB

```

R["Root LLM (Depth 0)"] --> C1["Sub-LLM: chunk 1 (D1)"]
R --> C2["Sub-LLM: chunk 2 (D1)"]
C1 --> S1a["Sub-LLM: sub 1a (D2)"]
C1 --> S1b["Sub-LLM: sub 1b (D2)"]
C2 --> S2a["Sub-LLM: sub 2a (D2)"]

```



Slide 6 — Why RLM beats everything

Four reasons:

1. **No context rot** — each LLM call sees only a small, focused piece, so "lost in the middle" disappears.
2. **Infinite scale** — 10K tokens or 100M tokens, same approach: just recurse deeper.
3. **Active exploration** — the model isn't passively fed data; it *chooses* what to look at and filters out the junk with code.
4. **Cost-effective** — throwing away irrelevant text with simple string operations is **free**. Only the genuinely useful ("gold") tokens get sent to the expensive GPU.

💡 **Remember:** Filtering with code is free; filtering with the transformer is expensive. RLM does the cheap filtering first.

Slide 7 — Results

The measured improvements (from the paper):

Setup	vs	Gain	Context capacity
RLM(GPT-5-mini)	Vanilla GPT-5	+34.2 points	100× beyond window
RLM(Qwen3-8B)	Base Qwen3-8B	+28.3% avg	Infinite recursion
RLM(Llama-3-70B)	Vanilla Llama-3	+19.5 points	Scales to 10M+ tokens

The headline result: **a small model using RLM beats a full-size model that just gets raw context** — and it handles inputs up to **100× larger** than the model's own window, at similar or lower cost.

Slide 8 — Claude Code vs Cursor (the real-world debate)

Now the slides switch to a real product comparison, because both these coding tools face the exact same "how do I search a huge codebase without drowning in context?" problem — and they answer it differently.

- **Claude Code** → **Agentic Search:** explores the codebase live using shell tools (`grep`, `glob`). **No pre-built index.**
- **Cursor** → **Indexed RAG:** builds a map of the code using **embeddings** stored in a **vector database**. Requires background indexing.

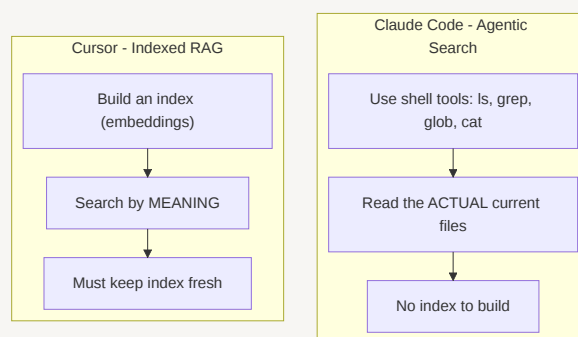
Both are billion-dollar products solving the same problem in opposite ways.

Quick definitions:

- **RAG** = Retrieval-Augmented Generation: fetch relevant snippets *first*, then feed only those to the model.
- **Embedding** = turning a piece of text into a list of numbers that captures its meaning, so "similar meaning" = "nearby numbers."
- **Vector database** = a store optimized for finding the "nearest" embeddings (closest meaning) fast.

flowchart TB

```
subgraph CC["Claude Code - Agentic Search"]
  CC1["Use shell tools: ls, grep, glob, cat"] --> CC2["Read the ACTUAL current files"]
  CC2 --> CC3["No index to build"]
end
subgraph CU["Cursor - Indexed RAG"]
  CU1["Build an index (embeddings)"] --> CU2["Search by MEANING"]
  CU2 --> CU3["Must keep index fresh"]
end
```



Slide 9 — Claude Code: Agentic Search

Its toolkit is just plain shell commands: `ls`, `glob`, `grep`, `cat`. **No embeddings. No index.**

Its process is like a detective asking questions in order:

1. **Discovery** — "What files exist?" → `ls`, `glob`
2. **Navigation** — "Where is this function?" → `grep`
3. **Inspection** — "What does it do?" → `cat`
4. **Context** — "What calls it?" → `grep` for references

Philosophy: active exploration over passive consumption. The model decides its own search strategy and keeps going until it understands. (Notice: this is the *exact same idea* as RLM — explore with tools instead of dumping everything in.)

Slide 10 — Why Agentic Search works

Three advantages:

1. **Precision** — code has exact symbols, and `grep` gives exact matches. Embeddings often return "fuzzy positives" (things that *seem* similar but are technically wrong).
2. **Freshness** — it reads the actual current file every time. No waiting to re-index after every save.
3. **Security & simplicity** — no index to maintain, and no sensitive code uploaded to a third-party vector database. Everything stays local.

The slide quotes Boris Cherny (Anthropic) on keeping their own sensitive codebase off third-party services.

Slide 11 — Cursor: the Indexed RAG approach

How Cursor makes indexing actually good:

- **Smart chunking** — uses **Tree-sitter** to parse code into its grammar tree, so it splits at clean boundaries (whole functions/classes) instead of cutting logic in half.
- **Incremental sync** — uses **Merkle trees** (a way to detect exactly which files changed) so it only re-embeds modified files, keeping the index fresh fast.
- **Team reuse** — teammates' code is ~92% identical, so a new member can reuse an existing index → near-zero setup.

- **Semantic search** — you can ask "Where do we handle rate limiting?" *without* knowing the function's name, because search is by meaning.

Reported result: semantic search improved accuracy by **12.5%**.

Slide 12 — Comparing the struggles

Neither approach is perfect:

Claude Code (Agentic)	Cursor (Indexed RAG)
Token burn — <code>grep</code> results pour into context, raising cost	Index staleness — a few-minute sync delay can mean stale files
Latency — playing "20 questions" with the filesystem is slow	Setup time — huge repos can take hours to index initially
Conceptual search — hard to find what you can't name exactly	Semantic noise — "similar but wrong" results can distract the model

💡 **Remember:** Agentic = precise but slow & pricey. Indexed = fast & conceptual but can be stale & fuzzy.

Slide 13 — The 2026 consensus & summary

- **The best teams use both:** Claude Code for deep, autonomous, multi-file work; Cursor for quick, visual, semantic discovery and fast UI iteration.
- **The core lesson:** *How a model relates to its context is the fundamental design decision of modern AI.* RLM, Claude Code, and Cursor are just three different answers to the same question.
- **Closing line:** *"Don't blindly stuff the context window."*

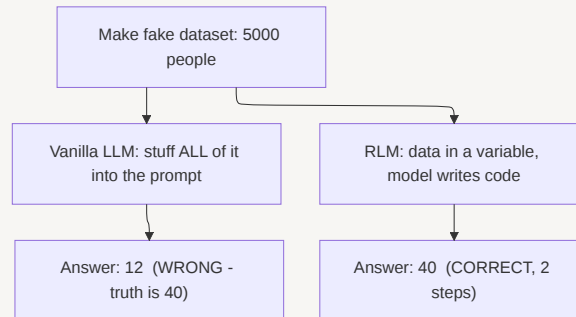
💡 **The whole class in one sentence:** Don't shove everything into the model — give it tools to fetch only what it needs.

Part C — The Colab Notebook, cell by cell

The notebook *builds* a tiny RLM from scratch so you can watch the idea work on real data. Here's what each piece does.

flowchart TB

```
D["Make fake dataset: 5000 people"] --> V["Vanilla LLM: stuff ALL of it into the prompt"]
V --> VR["Answer: 12 (WRONG - truth is 40)"]
D --> RL["RLM: data in a variable, model writes code"]
RL --> RR["Answer: 40 (CORRECT, 2 steps)"]
```



Cell 1 — Setup

Imports tools and defines one helper, `llm_call(prompt)`, which sends a prompt to a model (via **OpenRouter**, a service that gives access to many models) and returns the text reply. It picks `gemini-3-flash-preview` as both the root and sub model because it's cheap/free. A quick `print(llm_call("Say hello in one word."))` confirms the connection works → prints `Hello.`

Plain English: This cell is just "plug in the phone line to the AI and check there's a dial tone."

Cell 2 — Build the test dataset

Generates **5,000 fake people**, each line like: `Entry 11: Tina I., age 51, engineer, based in Cairo`. It also computes the **ground truth** the honest way (with plain Python) so we know the real answer: **40 engineers in Tokyo**. The whole thing is ~265,000 characters.

Why this task? Counting "engineers in Tokyo" is trivial for code but becomes a nightmare for an LLM when buried in 5,000 noisy lines. Perfect for exposing context rot.

Cell 3 — The vanilla attempt (the failure)

Stuffs **all 5,000 entries into a single prompt** and asks the LLM to count. Result:

```
Vanilla LLM answer: 12
Ground truth:      40
Correct?           ❌ No
```

This is **context rot live on screen** — the model can't reliably count things scattered through a giant blob of text. This failure is the entire motivation for everything that follows.

Cell 4 — Build the REPL (`RLMRepl` class)

This creates the "sandbox" where the model's code will run. Key ideas:

- The giant data is stored in a variable called `context` inside the sandbox — **never in the prompt**.
- It provides safe building blocks the model's code can use: `re`, `len`, `sum`, `print`, etc.
- `FINAL(answer)` — the model calls this to submit its final answer.
- `llm_query(query, chunk)` — lets the model spin up a **Sub-LLM** on a small chunk (this is the recursion).
- `execute(code)` runs the model's code, **captures whatever it prints**, and hands that text back. Output is truncated if it's too long, so the model never gets flooded.

A quick self-test peeks at a string, counts words with regex, and submits `FINAL(42)`

→ prints `✅ REPL works!`

Plain English: This is the desk where the data lives. The AI isn't allowed to grab the whole pile — it can only ask the desk to run little scripts and report back short answers.

Cell 5 — The system prompt

A block of instructions telling the model the rules of the game: *"You can't see `context` directly. Write Python to explore it. Use `print()` to see results. Use*

`llm_query()` for sub-tasks. Call `FINAL()` when done. Write only code, wrapped in a `python block`." This is what turns a normal chat model into an RLM agent.

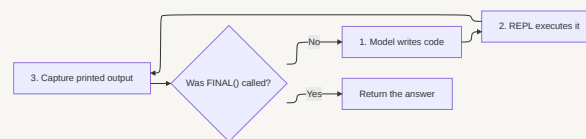
Cell 6 — The agent loop (`run_rlm`)

The engine. Two helpers:

- `extract_code(...)` — pulls the Python out of the model's reply (the model wraps code in `python ...`).
- `run_rlm(query, context, ...)` — the actual loop:
 1. Send the model the **question only** (plus how long the data is).
 2. Model writes code → `extract_code` grabs it.
 3. The REPL runs it; its printed output is captured.
 4. If `FINAL()` was called → stop and return the answer. Otherwise, feed the output back and **repeat** (up to a max number of iterations).

flowchart LR

```
W["1. Model writes code"] --> E["2. REPL executes it"]
E --> S["3. Capture printed output"]
S --> D{"Was FINAL() called?"}
D -->|"No"| W
D -->|"Yes"| F["Return the answer"]
```



Cell 7 — Run the RLM on the same task

Now run RLM on the identical "engineers in Tokyo" question. Watching the live trace:

- **Iteration 1:** model checks `len(context)` and peeks at `context[:1000]` to learn the format.

- **Iteration 2:** model writes a regex `r",\s+engineer,\s+based in Tokyo"`, counts matches, and calls `FINAL(40)`.

```
RLM answer:    40
Ground truth:  40
Correct?       ✓
Iterations:    2 | Sub-LLM calls: 0
```

Same model, same data, same question — but RLM gets it right. The model never pushed 5,000 entries through attention; it wrote ~3 lines of Python that ran in milliseconds.

(Regex = "regular expression" = a pattern for finding/matching text. `|s+` means "one or more spaces.")

Cell 8 — A harder dataset (restaurant reviews)

Counting only needs code. But what about tasks that need **understanding**, like sentiment? This cell builds **200 fake restaurant reviews** (e.g. `Review 2 [Ocean Plate]: The sushi was fantastic...`), some positive, some negative, some mixed. Code alone can't judge tone — this is where Sub-LLMs earn their keep.

Cell 9 — RLM on the semantic task (recursion in action)

The question: for each restaurant, give review count + overall sentiment + most-mentioned dish. The model's strategy:

1. **Code** groups the reviews by restaurant name (regex + a dictionary) — code is great at parsing/grouping.
2. For **each** restaurant, it calls `llm_query()` to ask a **Sub-LLM** to judge sentiment + top dish — LLMs are great at understanding.
3. It compiles all the results and calls `FINAL()`.

The trace shows **10 Sub-LLM calls** (one per restaurant). Crucially, **each Sub-LLM only saw ONE restaurant's reviews** — a tiny, clean context. No rot.



The core RLM division of labor:

- **Code** → parsing, grouping, counting, filtering (the boring exact stuff).

- **Sub-LLMs** → understanding, summarizing, judging tone (the fuzzy human stuff).
- **Root LLM** → the manager that orchestrates both.

Cell 10 — Scale test (does it actually scale?)

Now blow the data up to **20,000 entries (~1,076,295 characters)** — far past what you'd comfortably stuff into a prompt. RLM handles it in just **2 iterations**: peek at the format, then one regex count → `FINAL(146)`, matching the ground truth of **146**. The code ran in milliseconds and **none of the 20,000 entries ever entered the model's context window**.

⚠ **Tiny notebook typo to ignore:** the final `print` says "processes 2000 entries" — it's actually **20,000**. The number 146 is correct.

Part D — Master summary & what to remember

The three approaches side by side

	What the data does	How it searches	Best at	Weakness
Vanilla LLM	Goes <i>inside</i> the prompt	Reads it all at once	Tiny inputs	Context rot, hard size limit
RLM	Stays <i>outside</i> in a variable	Model writes code to slice/search/recurse	Massive inputs, exact tasks	Needs a code sandbox; many small calls
Cursor (Indexed RAG)	Pre-converted to embeddings	Search by <i>meaning</i>	Conceptual "where do we..." queries	Index can be stale/fuzzy
Claude Code (Agentic)	Stays on disk	Live shell tools (<code>grep</code>)	Precise, fresh, private	Slower, burns tokens

The five things to actually remember

1. **Context rot is the enemy.** Big blobs of text make models miss things, especially in the middle.

2. **RLM's trick:** keep the data *outside* the model as a variable; let the model write *code* to fetch only what it needs.
3. **Code is free, the transformer is expensive.** Filter junk with cheap string operations; spend GPU only on "gold" tokens.
4. **Recursion = infinite scale.** Split → split again → each helper sees a small piece → results flow back up.
5. **It's all one idea.** RLM, Claude Code, and Cursor are different answers to the same question: *how should a model relate to its context?* The wrong answer is always "just stuff the window."

Resources (from the notebook)

- Paper: arxiv.org/abs/2512.24601
- Official code: github.com/alexzhang13/r1m
- Prime Intellect blog: primeintellect.ai/blog/r1m
- Alex Zhang's blog: alexzhang13.github.io/blog/2025/r1m/